

US Patent & Trademark Office

Patent Public Search | Text View

United States Patent Application Publication

20250286883

Kind Code

A1

Publication Date

September 11, 2025

Inventor(s)

SON; Seok-Ho et al.

APPARATUS AND METHOD FOR MANAGING CLOUD COMPUTING INFRASTRUCTURE ACCESS BASED ON DYNAMIC PARALLEL NODES

Abstract

Disclosed herein are an apparatus and method for managing cloud computing infrastructure access based on dynamic parallel nodes. The apparatus for managing cloud computing infrastructure access based on dynamic parallel nodes may be configured to configure at least two of multiple compute nodes included in a computing infrastructure as representative nodes at preset intervals, validate, by the representative nodes, a user with respect to a remote command requested by the user for the computing infrastructure, and forward the remote command requested by a validated user to a target compute node included in the computing infrastructure.

Inventors: SON; Seok-Ho (Daejeon, KR), KANG; Dong-Jae (Daejeon, KR), KIM; Byoung-Seob (Sejong-si, KR), KIM; Soo-Young (Daejeon, KR), KIM; Yun-Kon (Daejeon, KR), BAE; Seung-Jo (Daejeon, KR), OH; Byeong-Thaek (Sejong-si, KR), JUNG; Young-Woo (Daejeon, KR)

Applicant: ELECTRONICS AND TELECOMMUNICATIONS RESEARCH INSTITUTE
(Daejeon, KR)

Family ID: 1000008338402

Assignee: ELECTRONICS AND TELECOMMUNICATIONS RESEARCH INSTITUTE
(Daejeon, KR)

Appl. No.: 18/970856

Filed: December 05, 2024

Foreign Application Priority Data

KR

10-2024-0031876

Mar. 06, 2024

Publication Classification

Int. Cl.: H04L9/40 (20220101)

U.S. Cl.:

CPC H04L63/083 (20130101); H04L63/108 (20130101);

Background/Summary

CROSS REFERENCE TO RELATED APPLICATION

[0001] This application claims the benefit of Korean Patent Application No. 10-2024-0031876, filed Mar. 6, 2024, which is hereby incorporated by reference in its entirety into this application.

BACKGROUND OF THE INVENTION

1. Technical Field

[0002] The present disclosure relates generally to cloud computing technology, and more particularly to technology for managing cloud computing infrastructure access based on dynamic parallel nodes.

2. Description of the Related Art

[0003] Through computing paradigms such as cloud computing and server virtualization, it is possible to configure a computing infrastructure composed of multiple on-demand compute nodes (e.g., various types of nodes such as virtual machines, containers, container runtimes, and computing instances). To remotely access the compute nodes configured in this way and execute commands thereon (i.e., access and command execution based on a protocol such as secure shell (SSH) or remote desktop protocol (RDP)), it is necessary to know an access address (such as an IP address, port number, etc.) and to have an appropriate authentication key according to the access protocol. Generally, when compute nodes do not need to be connected to an external system, such as the Internet, and to be exposed, the compute nodes are connected only to a private network (e.g., a private subnet) to block external networks, thus allowing only compute nodes connected to the private network to communicate with each other. However, the compute nodes present in the private network also need to allow an administrator to remotely access and control the compute nodes so as to perform settings, control, etc.

[0004] However, when a public IP address is exposed on the Internet, the corresponding compute node may continuously become a target for attacks, and an access address and an access key may be stolen even by an internal employee affiliated with the operator. In this case, a principal computing infrastructure present in a significant private network may be intimidated. Typically, when public cloud computing-based infrastructure services are used, a private computing infrastructure and a virtual private network (VPN) may be configured to create a secure channel similar to a dedicated network, thus making access to the compute node at a remote place and issuing commands to the compute node. In this case, this is safer than being directly exposed to the internet, but a cost associated with using a VPN or the dedicated network should be paid. Similarly, when one access address and access key are internally or externally stolen, the corresponding compute node may encounter attacks that are not known even by the administrator. In this way, conventional technology accesses and controls the compute node at a remote place, wherein the compute node has a large difficulty from the standpoint of remote security.

[0005] Meanwhile, U.S. Patent No. U.S. Pat. No. 11,323,477 entitled “Establishing secure connections to instances in private subnets of a cloud provider network” discloses a connection component for a cloud compute instance, in which a user establishes a Secure Shell (SSH)

connection with a compute instance executed in a private subnet within a virtual private network of a cloud provider network.

SUMMARY OF THE INVENTION

[0006] Accordingly, the present disclosure has been made keeping in mind the above problems occurring in the prior art, and an object of the present disclosure is to provision and manage a cloud computing infrastructure that utilizes various types of clouds, and improve secure access to the provisioned computing infrastructure.

[0007] Another object of the present disclosure is to dynamically extend nodes depending on a cloud access level and enhance security by mutually validating the nodes.

[0008] A further object of the present disclosure is to allow an authenticated and authorized user to freely access nodes and request the nodes to process commands at an external place without passing through a Virtual Private Network (VPN) or a dedicated network.

[0009] In accordance with an aspect of the present disclosure to accomplish the above objects, there is provided an apparatus for managing cloud computing infrastructure access based on dynamic parallel nodes, including one or more processors, and memory configured to store at least one program that is executed by the one or more processors, wherein the at least one program is configured to configure at least two of multiple compute nodes included in a computing infrastructure as representative nodes at preset intervals, validate, by the representative nodes, a user with respect to a remote command requested by the user for the computing infrastructure, and forward the remote command requested by a validated user to a target compute node included in the computing infrastructure.

[0010] The at least one program may be configured to configure the representative nodes in a multi-layer structure.

[0011] The at least one program may be configured to determine that validation of the user has succeeded only when the remote command is forwarded to the target compute node through the representative nodes configured in the multi-layer structure.

[0012] The at least one program may be configured to set a time window for an accepted request time interval for the remote command with respect to the representative nodes.

[0013] The at least one program may be configured to determine that validation of the user has succeeded only when the remote command is requested from representative nodes within the time window.

[0014] The at least one program may be configured to set an accepted number of requests corresponding to the remote command requested from the representative nodes within the time window.

[0015] The at least one program may be configured to determine that validation of the user has succeeded only when a remote command corresponding to the accepted number of requests is requested from the representative nodes within the time window.

[0016] The at least one program may be configured to set a request sequence related to an order in which the remote command is requested from the representative nodes.

[0017] The at least one program may be configured to determine that validation of the user has succeeded only when the order in which the remote command is requested from the representative nodes depending on the request sequence matches the set request sequence.

[0018] The at least one program may be configured to convert the remote command of the user into a command format corresponding to the target compute node using information stored in a command mapping database.

[0019] In accordance with an aspect of the present disclosure to accomplish the above objects, there is provided a method for managing cloud computing infrastructure access based on dynamic parallel nodes, including configuring at least two of multiple compute nodes included in a computing infrastructure as representative nodes at preset intervals, validating, by the representative nodes, a user with respect to a remote command requested by the user for the

computing infrastructure, and forwarding the remote command requested by a validated user to a target compute node included in the computing infrastructure.

[0020] Configuring as the representative nodes may include configuring the representative nodes in a multi-layer structure.

[0021] Validating the user may include determining that validation of the user has succeeded only when the remote command is forwarded to the target compute node through the representative nodes configured in the multi-layer structure.

[0022] Configuring as the representative nodes may include setting a time window for an accepted request time interval for the remote command with respect to the representative nodes.

[0023] Validating the user may include determining that validation of the user has succeeded only when the remote command is requested from representative nodes within the time window.

[0024] Configuring as the representative nodes may further include setting an accepted number of requests corresponding to the remote command requested from the representative nodes within the time window.

[0025] Validating the user may include determining that validation of the user has succeeded only when a remote command corresponding to the accepted number of requests is requested from the representative nodes within the time window.

[0026] Configuring as the representative nodes may include setting a request sequence related to an order in which the remote command is requested from the representative nodes.

[0027] Validating the user may include determining that validation of the user has succeeded only when the order in which the remote command is requested from the representative nodes depending on the request sequence matches the set request sequence.

[0028] Forwarding the remote command may include converting the remote command of the user into a command format corresponding to the target compute node using information stored in a command mapping database.

Description

BRIEF DESCRIPTION OF THE DRAWINGS

[0029] The above and other objects, features and advantages of the present disclosure will be more clearly understood from the following detailed description taken in conjunction with the accompanying drawings, in which:

[0030] FIG. 1 is a diagram illustrating a multi-cloud computing infrastructure according to an embodiment of the present disclosure;

[0031] FIG. 2 is a block diagram illustrating a system for managing cloud computing infrastructure access based on dynamic parallel nodes according to an embodiment of the present disclosure;

[0032] FIG. 3 is a block diagram illustrating an apparatus for managing cloud computing infrastructure access based on dynamic parallel nodes according to an embodiment of the present disclosure;

[0033] FIG. 4 is a diagram illustrating a multi-layer bastion group according to an embodiment of the present disclosure;

[0034] FIG. 5 is a diagram illustrating a process in which ephemeral bastions interact with each other for user request verification according to an embodiment of the present disclosure;

[0035] FIG. 6 is a diagram illustrating a time window validation scheme according to an embodiment of the present disclosure;

[0036] FIG. 7 is a diagram illustrating a request sequence validation scheme according to an embodiment of the present disclosure;

[0037] FIG. 8 is a diagram illustrating an OS agnostic command conversion process according to an embodiment of the present disclosure;

[0038] FIG. 9 is an operation flowchart illustrating a method for managing cloud computing infrastructure access based on dynamic parallel nodes according to an embodiment of the present disclosure; and

[0039] FIG. 10 is a diagram illustrating a computer system according to an embodiment of the present disclosure.

DESCRIPTION OF THE PREFERRED EMBODIMENTS

[0040] The present disclosure will be described in detail below with reference to the accompanying drawings. Repeated descriptions and descriptions of known functions and configurations which have been deemed to make the gist of the present disclosure unnecessarily obscure will be omitted below. The embodiments of the present disclosure are intended to fully describe the present disclosure to a person having ordinary knowledge in the art to which the present disclosure pertains. Accordingly, the shapes, sizes, etc. of components in the drawings may be exaggerated to make the description clearer.

[0041] In the present specification, it should be understood that terms such as “include” or “have” are merely intended to indicate that features, numbers, steps, operations, components, parts, or combinations thereof are present, and are not intended to exclude the possibility that one or more other features, numbers, steps, operations, components, parts, or combinations thereof will be present or added.

[0042] Hereinafter, embodiments of the present disclosure will be described in detail with reference to the attached drawings.

[0043] FIG. 1 is a diagram illustrating a multi-cloud computing infrastructure according to an embodiment of the present disclosure.

[0044] Referring to FIG. 1, a provisioned multi-cloud computing infrastructure according to an embodiment of the present disclosure is illustrated.

[0045] The multi-cloud computing infrastructure is implemented such that multiple infrastructures including a single cloud and heterogeneous clouds are configured in a single form.

[0046] In the multi-cloud computing infrastructure, networks (VPC and Subnet), compute nodes, a security group, a disk of compute nodes, etc. may be freely and dynamically configured. SubGroup may be logically configured in the form of a cluster including compute nodes and bastion nodes mutually connected to each other through an internal network.

[0047] Each of the compute nodes and the bastion nodes may include a root disk (RootDisk) and a data disk (DataDisk).

[0048] Further, the multi-cloud computing infrastructure may configure a representative node for an interface that is connected to the outside over the Internet or that can be connected to a specific location through a VPN or the like.

[0049] The representative node may be referred to as a bastion node. After the administrator accesses the multi-cloud computing infrastructure through the bastion node, the administrator may newly access another compute node connected through a subnet within the bastion node, and may then perform control. A system which performs this action may be referred to as a jump host.

[0050] The multi-cloud computing infrastructure may support the on-demand creation of computing resources as needed.

[0051] Here, when the cloud-based computing infrastructure is configured, the multi-cloud computing infrastructure may configure a bastion node in which a specific private infrastructure that is exposed to the external Internet by allocating a public IP address or that has an administrator through a Virtual Private Network (VPN) service is connected to a dedicated secure network.

[0052] The provisioned multi-cloud computing infrastructure may provide status management and lifecycle control in an integrated manner for a computing infrastructure unit, a subgroup unit, an individual node unit, etc.

[0053] The provisioned multi-cloud computing infrastructure may manage the lifecycle status of the computing infrastructure such as an operating state, stop state, and a termination state.

[0054] The provisioned multi-cloud computing infrastructure may determine whether the lifecycle of each compute node is identical to the lifecycle of the actual computing instance managed by the cloud service provider, synchronize the lifecycles, and correct errors.

[0055] Further, the provisioned multi-cloud computing infrastructure may include functions of performing and managing stop, resume, restart, and terminate operations, and the like.

[0056] FIG. 2 is a block diagram illustrating a system for managing cloud computing infrastructure access based on dynamic parallel nodes according to an embodiment of the present disclosure.

[0057] Referring to FIG. 2, an apparatus **100** for managing cloud computing infrastructure access based on dynamic parallel nodes (also referred to as a computing infrastructure manager **100**) may dynamically designate bastion nodes in a bastion node group **10** so as to remotely access a compute node or a service connected to a private network in a computing infrastructure distributed by a user, or remotely issue a command to the compute node or the service.

[0058] Here, the apparatus **100** for managing cloud computing infrastructure access based on dynamic parallel nodes may configure the bastion node group **10** to form ephemeral bastion nodes without manually placing only one bastion node.

[0059] Here, the apparatus **100** for managing cloud computing infrastructure access based on dynamic parallel nodes may cause a malicious attacker to lose an attack target by dynamically controlling the number of bastion nodes included in the bastion node group **10** as needed, thus maximizing access security.

[0060] Here, the apparatus **100** for managing cloud computing infrastructure access based on dynamic parallel nodes may mutually validate whether the bastion node group **10** receives the same request from an administrator **20** or the attacker **30**, and may process a requested remote command to prevent the requested remote command from being executed when not all bastion nodes and their security keys are known, thus improving security.

[0061] Also, each bastion node maintained for a long time is more likely to continuously undergo external attacks and intrusion attempts.

[0062] Accordingly, the apparatus **100** for managing cloud computing infrastructure access based on dynamic parallel nodes may detect anomalies in old bastion nodes and automatically discard the corresponding nodes.

[0063] Here, the apparatus **100** for managing cloud computing infrastructure access based on dynamic parallel nodes may neutralize the existing attacks by the attacker **30** by adding an entirely new bastion node, and may prevent the attacker **30** from finding an attack target to lose the attack target itself by changing an endpoint address.

[0064] Here, the apparatus **100** for managing cloud computing infrastructure access based on dynamic parallel nodes may configure a time window for a request acceptance time point at which only an authenticated and authorized user is recognized.

[0065] Here, when the accurate request acceptance time point is not recognized, the apparatus **100** for managing cloud computing infrastructure access based on dynamic parallel nodes may securely protect private compute nodes therein so that the remote command is not propagated to the private compute nodes.

[0066] Here, the apparatus **100** for managing cloud computing infrastructure access based on dynamic parallel nodes may designate a specific time point which is known only to allowed users when higher level security is required.

[0067] Here, the apparatus **100** for managing cloud computing infrastructure access based on dynamic parallel nodes may be configured to determine whether the same request has been input as many times as the number of bastion nodes within a specific time window and to check whether the requests are made sequentially in the designated order.

[0068] Here, the apparatus **100** for managing cloud computing infrastructure access based on dynamic parallel nodes may inject laws to be kept in a time dimension.

[0069] Here, the apparatus **100** for managing cloud computing infrastructure access based on

dynamic parallel nodes may deny all requests from the attacker **30** who is unaware of the sequence and time frames, and may recognize whether the attacker **30** has made an attack.

[0070] In addition, in the apparatus **100** for managing cloud computing infrastructure access based on dynamic parallel nodes, the compute nodes within the infrastructure may utilize an Operating System (OS) that becomes a basis, and the version of the OS.

[0071] Here, in the apparatus **100** for managing cloud computing infrastructure access based on dynamic parallel nodes, each bastion node or a computing infrastructure manager may convert the command so that the command is OS agnostic.

[0072] FIG. **3** is a block diagram illustrating an apparatus for managing cloud computing infrastructure access based on dynamic parallel nodes according to an embodiment of the present disclosure. FIG. **4** is a diagram illustrating a multi-layer bastion group according to an embodiment of the present disclosure. FIG. **5** is a diagram illustrating a process in which ephemeral bastions interact with each other for user request verification according to an embodiment of the present disclosure. FIG. **6** is a diagram illustrating a time window validation scheme according to an embodiment of the present disclosure. FIG. **7** is a diagram illustrating a request sequence validation scheme according to an embodiment of the present disclosure. FIG. **8** is a diagram illustrating an OS agnostic command conversion process according to an embodiment of the present disclosure.

[0073] Referring to FIG. **3**, the apparatus for managing cloud computing infrastructure access based on dynamic parallel nodes according to the embodiment of the present disclosure may be configured in the form of a computing infrastructure manager or a computing infrastructure access manager (i.e., this structure is only an embodiment, and there is no need to implement the method through the corresponding structure).

[0074] The apparatus for managing cloud computing infrastructure access based on dynamic parallel nodes according to the embodiment of the present disclosure may include a computing infrastructure (infra) provisioning and management unit, a bastion node configuration unit, a validation logic injection unit, a bastion node retention policy management unit, an infrastructure access management unit, a user authentication unit, a remote command request handler, an OS agnostic command conversion unit, and a remote command request logging unit.

[0075] In the computing infrastructure provisioning and management unit, a computing infrastructure manager may provision computing resources and the infrastructure in an integrated manner based on Application Programming Interfaces (APIs) of multiple heterogeneous clouds, and may perform integrated control and management on the lifecycle of the infrastructure.

[0076] The bastion node configuration unit may designate a specific compute node as a bastion node so as to access a provisioned or configured computing infrastructure and support processing of remote commands thereon.

[0077] The bastion node configuration unit may configure at least two of the multiple compute nodes included in the computing infrastructure as bastion nodes at preset intervals.

[0078] Here, the bastion node configuration unit may configure a new compute node for a bastion purpose, and may designate the compute node as a bastion node.

[0079] The bastion node may be dynamically set at the time of creating an infrastructure, or may also be set after the infrastructure has been created.

[0080] Here, when the network IP address of a user who uses the bastion node is clearly defined, the bastion node configuration unit may specify a security group, and may more precisely restrict allowable transmission/reception network bands and access ports.

[0081] Each bastion node may be configured to be connected to compute nodes included only in a private subnet.

[0082] Here, the bastion node configuration unit may access the internal compute node through the bastion node (jump host) or request the internal compute node to process a remote command.

[0083] Here, the bastion node configuration unit may dynamically designate multiple bastion nodes, and may designate a collection of bastion nodes as a bastion group.

[0084] Here, the bastion node configuration unit may configure bastion nodes as disposable temporary (ephemeral) bastion nodes.

[0085] A bastion node maintained for a long time is more likely to continuously undergo external attacks and intrusion attempts.

[0086] Accordingly, the bastion node configuration unit may detect anomalies to automatically discard the corresponding bastion node itself.

[0087] Here, the bastion node configuration unit may neutralize existing attacks made by an attacker by adding an entirely new bastion node, and may prevent the attacker from finding an attack target to lose the attack target itself by changing an endpoint address.

[0088] Here, the bastion node configuration unit may terminate the current bastion node and designate another node as a bastion node so as to cause the attacker to lose the attack target itself.

[0089] Here, the bastion node configuration unit may change the bastion node by changing the state or shape of the bastion node in such a way as to newly reissue and set only an endpoint related to access such as IP or port.

[0090] The bastion node retention policy unit may set a time point at which the existing bastion node is removed from the bastion group and a new bastion node is configured based on the system or user policy, and may set the reference number of bastion nodes in the bastion group.

[0091] The retention policy may be defined and dynamically executed in consideration of the maintenance cost of the bastion node and required security level thereof.

[0092] For example, when higher security is required, the bastion node retention policy management unit may designate a policy so as to discard all bastion nodes that have processed at least one remote command and configure new bastion nodes.

[0093] Further, the bastion node retention policy management unit may predict the sign of an attack to probabilistically determine the attack, discard only a related bastion node, and newly reconstruct a bastion node.

[0094] In addition, the bastion node retention policy management unit may discard the corresponding bastion node and update bastion nodes even when the performance of the corresponding bastion node is deteriorated through monitoring, as well as a security-related element.

[0095] Furthermore, the bastion node retention policy management unit may declaratively define the number of bastion nodes to be maintained at a specific number.

[0096] Here, the bastion node retention policy management unit may be configured to dynamically designate the appropriate number of nodes rather than the specific number.

[0097] Similarly, as the number of bastion nodes increases, security may be maximized.

[0098] Here, as the number of nodes increases, cost may increase, and thus the bastion node retention policy management unit may adjust the size of nodes or may designate the existing compute node as a bastion node to allow the existing compute node to perform a dual function so as to reduce the cost.

[0099] Furthermore, the bastion node retention policy management unit may determine the level of intimidation, and may also designate a dynamic policy of temporarily increasing and then decreasing the number of bastion nodes only when the level of intimidation is high.

[0100] Here, the bastion node retention policy management unit may hold access information/security key pairs of all bastion nodes depending on validation logic. From the standpoint of an attacker, there is a need to acquire multiple pieces of access information, thus making it extremely difficult to attempt intrusions.

[0101] The validation logic injection unit may configure the bastion nodes as in a multi-layer structure, thus further maximizing security.

[0102] Referring to FIG. 4, it can be seen that a multi-layer bastion group according to an embodiment of the present disclosure is illustrated.

[0103] Here, in the case where the bastion group is configured as a multi-layer group, the

validation logic injection unit may execute validation logic for forwarding commands to a private compute node only when the configuration of the layers is recognized.

[0104] Here, the validation logic injection unit may determine that validation has succeeded only when the remote command is forwarded to the target compute node via the bastion nodes configured in the multi-layer structure.

[0105] Here, when the multi-layer bastion group is configured, the corresponding bastion nodes may execute special validation logic for checking whether the remote command of the user and access by the user are legitimate.

[0106] Here, the validation logic may also be configured through settings when the bastion nodes are provisioned.

[0107] Here, the validation logic may be variously implemented such as initialization setting through cloud-init, setting to previously included validation logic, creation of compute nodes using a previously set image, and setting through distribution of a dedicated agent.

[0108] Here, the validation logic may be injected and set into a user request when the user request occurs (for example, the validation logic may be added as a command to an SSH command and then be forwarded to the corresponding bastion node).

[0109] The bastion nodes may mutually validate the user with respect to the remote command requested by the user for a computing infrastructure.

[0110] Here, the validation logic injection unit may process authentication of the remote command desired by the user to be executed on the compute node and validation of a legitimate request from an authorized user in a multidimensional manner.

[0111] Referring to FIG. 5, a process in which ephemeral bastions interact with each other for user request validation according to an embodiment of the present disclosure is illustrated.

[0112] A user (client) **20** may directly forward a remote command such as SSH to all bastion nodes **10**, thus allowing an internal specific compute node or a compute node group to execute the corresponding remote command.

[0113] Here, each of the bastion nodes **10** may validate whether the user **20** is a legitimate user and a requested command contains any malicious intent.

[0114] All bastion nodes **10** included in the bastion group may check whether they have received the same request while exchanging requests received from the user **20** with each other.

[0115] Here, when the bastion nodes **10** do not receive the same request, the bastion nodes **10** may assume that the user is not an authenticated user or that the request is an intrusion request rather than a normal request, and may drop the request or intentionally return a falsified response value so as to confuse the attacker.

[0116] In order for the attacker to intrude into compute nodes, the attacker needs to know access information/access key pairs of all bastions and compute nodes that are the target of attack. If even one of the access information/access key pairs is incorrect, the corresponding user is detected as being an attacker.

[0117] Referring to FIG. 6, it can be seen that a time window validation scheme according to an embodiment of the present disclosure is illustrated.

[0118] The time window validation scheme may validate whether the same request from a user has been received through all bastion nodes within a time window designated by the user or the system.

[0119] When no request is received within the corresponding time window, all of the bastion nodes may determine that there is a potential threat of attack, and may take suitable security measures.

[0120] Here, the validation logic injection unit may designate the time window.

[0121] Here, the validation logic injection unit may set the time window for a request acceptance time period corresponding to the remote command with respect to the bastion nodes.

[0122] Here, the validation logic injection unit may determine that validation of the user has succeeded only when the remote command is requested from the bastion nodes within the time window.

[0123] Here, the validation logic injection unit may set the accepted number of requests for the remote command for the bastion nodes within the time window.

[0124] Here, the validation logic injection unit may determine that validation of the user has succeeded only when the remote command corresponding to the accepted number of requests is requested from the bastion nodes within the time window.

[0125] Here, the validation logic injection unit may dynamically set the size and location of the time window in consideration of a desired security level, a current security threat, etc.

[0126] For example, when it is currently analyzed that there are a number of security threats, the validation logic injection unit may set the size of the time window to be very small and frequently change the location of the time window. By means of this operation, a higher level of security validation in an additional dimension (time on) may be performed.

[0127] As shown in FIG. 6, it may be determined that validation is effective only when the request is called to a valid number of bastion nodes during the interval of a preset valid time window (acceptance time window).

[0128] When the number of called bastion nodes is less than the preset number of bastion nodes even in the valid time window in which the request for the bastion nodes is accepted, it is determined that validation is not effective, and the corresponding request may be dropped or, alternatively, a falsified response value may be intentionally returned to the attacker to confuse the attacker.

[0129] Referring to FIG. 7, a request sequence validation scheme according to an embodiment of the present disclosure may be illustrated.

[0130] The request sequence validation scheme may validate whether an authenticated and authorized user has called a remote command request to bastion nodes according to a set order.

[0131] Here, the validation logic injection unit may set a request sequence related to the order in which the remote command is requested from the bastion nodes.

[0132] Here, the validation logic injection unit may determine that validation of the user has succeeded only when the order in which the remote command is requested from the bastion nodes according to the request sequence matches the set request sequence.

[0133] Here, the validation logic injection unit may set a request sequence related to the order in which the remote command is requested from the bastion nodes within the time window.

[0134] Here, the validation logic injection unit may determine that validation of the user has succeeded only when the order in which the remote command is requested from the bastion nodes according to the request sequence matches the set request sequence within the time window.

[0135] Because the attacker does not know the order in which bastion nodes present in the bastion group are to be called, it is impossible to intrude into the corresponding bastion node even if access information is merely stolen.

[0136] As shown in FIG. 7, the valid request sequence is 7, 2, 3, 4, 1, 6, and 5, and it can be seen that validation has succeeded only when bastion nodes with seven numbers designated are to be called in the order of 7, 2, 3, 4, 1, 6, and 5.

[0137] The order of 1, 2, 3, and 4 or the order of 6, 2, 5, 3, 4, 1, and 7 may be determined to be an invalid request sequence, with the result that the corresponding request may be dropped or, alternatively, a falsified response value may be intentionally returned to the attacker so as to confuse the attacker.

[0138] After the computing infrastructure has been provisioned, the infrastructure access management unit may provide access information of compute nodes included in the computing infrastructure to the user.

[0139] The access information may include access addresses, access methods, access keys, bastion information, etc. of all compute nodes including the bastion nodes.

[0140] A user authentication unit may allow only a legitimate user to request access information through authentication and authorization. When the user receives the access information, SSH or

the like may be performed based on the access information.

[0141] A compute node included only in a private network needs to be remotely accessed or needs to transmit a command by making a jump host connection (e.g., a scheme for adding an SSH command to the SSH to overlap the SSH) through the corresponding bastion node using a designated method.

[0142] A remote command request handler may forward a remote command requested by the validated user to a target compute node included in the computing infrastructure.

[0143] Here, the remote command request handler may forward the command depending on a designated validation method so as to perform validation in the bastion node instead of the bastion node, or so as to prevent any problems from occurring at the time of performing validation between bastion nodes.

[0144] Here, the remote command request handler may be configured such that, when the command is forwarded, an OS agnostic command conversion unit performs command conversion as needed.

[0145] The OS agnostic command conversion unit may perform command conversion so that the same command is executed on compute nodes having different OSs.

[0146] Here, the OS agnostic command conversion unit may convert the remote command from the user in the format of a command corresponding to a target compute node using information stored in a command mapping database (DB).

[0147] Finally, the remote command request handler may handle the request from the user to allow each bastion node to forward the request to the corresponding compute node by forwarding the request to all related bastion nodes (forward remote commands).

[0148] In the case where compute nodes are based on different OSs or environments when the user requests a remote command from compute nodes in the form of a command working on a specific OS, there is a high likelihood that the corresponding command will not work.

[0149] Referring to FIG. 8, an OS agnostic command conversion process according to an embodiment of the present disclosure is illustrated.

[0150] The OS agnostic command conversion unit may configure an OS agnostic command mapping DB from key commands for respective OSs, and may convert the command of a source OS into the command of a target OS based on information stored in the OS agnostic command mapping DB.

[0151] Here, the OS agnostic command conversion unit may provide a command recommender (simulator) for collecting and analyzing information in the OS agnostic command mapping DB, an external DB or the Internet web and converting the information into a suitable command.

[0152] In particular, the command simulator may configure an optimal command and then set up temporary simulators for respective OS environments to proactively validate whether a recommended command can work.

[0153] Here, the OS agnostic command conversion unit may allocate the validated command to each compute node, thus allowing all compute nodes to execute the corresponding command to be OS agnostic.

[0154] Here, the OS agnostic command conversion unit may allow the user to confirm the converted command in advance and forward the command to the corresponding compute node if necessary.

[0155] As shown in FIG. 8, it can be seen that the command recommender (simulator) converts Ubuntu 18.04 command into Ubuntu 22.04, Ubuntu 18.04, Windows 2000, and Debian **10** commands based on the OS agnostic command mapping DB.

[0156] A feedback handler may update the OS agnostic command mapping DB by feeding back the results of command conversion and the results of command output (stdout, stderr, or the like).

[0157] A remote command request logging unit may store and manage the request and result of the remote command by the user, the result of validation, etc. through logging.

[0158] Here, the remote command request logging unit may analyze and utilize the degree of system attack threats or the like for various policy establishment.

[0159] FIG. 9 is an operation flowchart illustrating a method for managing cloud computing infrastructure access based on dynamic parallel nodes according to an embodiment of the present disclosure.

[0160] Referring to FIG. 9, the method for managing cloud computing infrastructure access based on dynamic parallel nodes according to an embodiment of the present disclosure may configure bastion nodes at step S210.

[0161] That is, at step S210, at least two of the multiple compute nodes included in a computing infrastructure may be configured as bastion nodes at preset intervals.

[0162] Here, step S210 may be performed to designate a specific compute node as a bastion node so as to access a provisioned or configured computing infrastructure and support processing of remote commands thereon.

[0163] Here, step S210 may be performed to configure a new compute node for a bastion purpose and designate the new compute node as the bastion node.

[0164] The bastion node may be dynamically set at the time of creating an infrastructure, or may also be set after the infrastructure has been created.

[0165] Here, step S210 may be performed to, when the network IP address of a user who uses the bastion node is clearly defined, specify a security group, and more precisely restrict allowable transmission/reception network bands and access ports.

[0166] Each bastion node may be configured to be connected to compute nodes included only in a private subnet.

[0167] Here, step S210 may be performed to access the internal compute node through the bastion node (jump host) or request the internal compute node to process a remote command.

[0168] Here, step S210 may be performed to dynamically designate multiple bastion nodes, and to designate a collection of bastion nodes as a bastion group.

[0169] Here, step S210 may be performed to configure bastion nodes as disposable temporary (ephemeral) bastion nodes.

[0170] A bastion node maintained for a long time is more likely to continuously undergo external attacks and intrusion attempts.

[0171] Accordingly, step S210 may be performed to detect anomalies to automatically discard the corresponding bastion node itself.

[0172] Here, step S210 may be performed to neutralize existing attacks made by an attacker by adding an entirely new bastion node, and prevent the attacker from finding an attack target to lose the attack target itself by changing an endpoint address.

[0173] Here, step S210 may be performed to terminate the current bastion node and designate another node as a bastion node so as to cause the attacker to lose the attack target itself.

[0174] Here, step S210 may be performed to change the bastion node by changing the state or shape of the bastion node in such a way as to newly reissue and set only an endpoint related to access such as IP or port.

[0175] Here, step S210 may be performed to set a time point at which the existing bastion node is removed from the bastion group and a new bastion node is configured based on the system or user policy, and set the reference number of bastion nodes in the bastion group.

[0176] The retention policy may be defined and dynamically executed in consideration of the maintenance cost of the bastion node and required security level thereof.

[0177] Here, step S210 may be performed to, when higher security is required, designate a policy so as to discard all bastion nodes that have processed at least one remote command and configure new bastion nodes.

[0178] Here, at step S210, the sign of an attack may be predicted to probabilistically determine the attack, only a related bastion node may be discarded, and a bastion node may be newly

reconstructed.

[0179] Here, at step **S210**, it may be possible to discard the corresponding bastion node and thereafter update the bastion node even when the performance of the bastion node is deteriorated through monitoring, as well as a security-related element.

[0180] Here, at step **S210**, the number of bastion nodes may be declaratively defined to be maintained at a specific number.

[0181] Here, step **S210** may be configured to dynamically designate the appropriate number of nodes rather than the specific number.

[0182] Similarly, as the number of bastion nodes increases, security may be maximized.

[0183] Here, as the number of nodes increases, cost may increase, and thus step **S210** may be performed to adjust the size of nodes or designate the existing compute node as a bastion node to allow the existing compute node to perform a dual function so as to reduce the cost.

[0184] Here, step **S210** may be performed to determine the level of intimidation, and also designate a dynamic policy of temporarily increasing and then decreasing the number of bastion nodes only when the level of intimidation is high.

[0185] Here, step **S210** may be performed to hold access information/security key pairs of all bastion nodes depending on validation logic. From the standpoint of an attacker, there is a need to acquire multiple pieces of access information, thus making it extremely difficult to attempt intrusions.

[0186] Here, step **S210** may be performed to configure the bastion nodes as in a multi-layer structure, thus further maximizing security.

[0187] Referring to FIG. 4, it can be seen that a multi-layer bastion group according to an embodiment of the present disclosure is illustrated.

[0188] Here, step **S210** may be performed to, in the case where the bastion group is configured as a multi-layer group, create validation logic for forwarding commands to a private compute node only when the configuration of the layers is recognized.

[0189] Here, at step **S210**, when the multi-layer bastion group is configured, the corresponding bastion nodes may execute special validation logic for checking whether the remote command of the user and access by the user are legitimate.

[0190] Here, the validation logic may also be configured through settings when the bastion nodes are provisioned.

[0191] Here, the validation logic may be variously implemented such as initialization setting through cloud-init, setting to previously included validation logic, creation of compute nodes using a previously set image, and setting through distribution of a dedicated agent.

[0192] Here, the validation logic may be injected and set into a user request when the user request occurs (for example, the validation logic may be added as a command to an SSH command and then be forwarded to the corresponding bastion node).

[0193] Here, step **S210** may be performed to, after the computing infrastructure has been provisioned, provide access information of compute nodes included in the computing infrastructure to the user.

[0194] The access information may include access addresses, access methods, access keys, bastion information, etc. of all compute nodes including the bastion nodes.

[0195] Here, step **S210** may be performed to allow only a legitimate user to request access information through authentication and authorization. When the user receives the access information, SSH or the like may be performed based on the access information.

[0196] A compute node included only in a private network needs to be remotely accessed or needs to transmit a command by making a jump host connection (e.g., a scheme for adding an SSH command to the SSH to overlap the SSH) through the corresponding bastion node using a designated method.

[0197] Here, step **S210** may be performed to set the time window for a request acceptance time

period corresponding to the remote command to the bastion nodes.

[0198] Here, step **S210** may be performed to set the accepted number of requests for the remote command for the bastion nodes within the time window.

[0199] Here, step **S210** may be performed to set a request sequence related to the order in which the remote command is requested from the bastion nodes.

[0200] Here, step **S210** may be performed to set a request sequence related to the order in which the remote command is requested from the bastion nodes within the time window.

[0201] Further, the method for managing cloud computing infrastructure access based on dynamic parallel nodes according to the embodiment of the present disclosure may verify the user who requests the remote command at step **S220**.

[0202] That is, at step **S220**, the bastion nodes may validate the user for the remote command requested by the user for the computing infrastructure.

[0203] Here, step **S220** may be performed to process authentication of the remote command desired by the user to be executed on the compute node and validation of a legitimate request from an authorized user in a multidimensional manner.

[0204] Here, at step **S220**, in the case where the bastion nodes are configured in a multi-layer structure, it may be determined that validation has succeeded only when the remote command is forwarded to a target compute node through the bastion nodes configured in the multi-layer structure.

[0205] Referring to FIG. 5, a process in which ephemeral bastions interact with each other for user request validation according to an embodiment of the present disclosure is illustrated.

[0206] At step **S220**, a user (client) **20** may directly forward a remote command such as SSH to all bastion nodes **10**, thus allowing an internal specific compute node or a compute node group to execute the corresponding remote command.

[0207] Here, step **S220** may be performed to validate whether the user **20** is a legitimate user and a requested command contains any malicious intent.

[0208] Here, at step **S220**, all bastion nodes **10** included in the bastion group may check whether they have received the same request while exchanging requests received from the user **20** with each other.

[0209] Here, at step **S220**, when the bastion nodes **10** do not receive the same request, the bastion nodes **10** may assume that the user is not an authenticated user or that the request is an intrusion request rather than a normal request, and may drop the request or intentionally return a falsified response value so as to confuse the attacker.

[0210] In order for the attacker to intrude into compute nodes, the attacker needs to know access information/access key pairs of all bastions and compute nodes that are the target of attack. If even one of the access information/access key pairs is incorrect, the corresponding user is detected as being an attacker.

[0211] Referring to FIG. 6, it can be seen that a time window validation scheme according to an embodiment of the present disclosure is illustrated.

[0212] The time window validation scheme may validate whether the same request from a user has been received through all bastion nodes within a time window designated by the user or the system.

[0213] When no request is received within the corresponding time window, all of the bastion nodes may determine that there is a potential threat of attack, and may take suitable security measures.

[0214] Here, at step **S220**, the time window may be designated.

[0215] Here, step **S220** may be performed to dynamically set the size and location of the time window in consideration of a desired security level, a current security threat, etc.

[0216] Here, step **S220** may be performed to determine that validation of the user has succeeded only when the remote command is requested from the bastion nodes within the time window.

[0217] Here, step **S220** may be performed to determine that validation of the user has succeeded only when the remote command corresponding to the accepted number of requests is requested

from the bastion nodes within the time window.

[0218] Here, step **S220** may be performed to, when it is currently analyzed that there are a number of security threats, set the size of the time window to be very small and frequently change the location of the time window. By means of this operation, a higher level of security validation in an additional dimension (time) may be performed.

[0219] As shown in FIG. 6, step **S220** may be performed to determine that validation is effective only when the request is called to a valid number of bastion nodes during the interval of a preset valid time window (acceptance time window).

[0220] Here, at step **S220**, when the number of called bastion nodes is less than the preset number of bastion nodes even in the valid time window in which the request for the bastion nodes is accepted, it is determined that validation is not effective, and the corresponding request may be dropped or, alternatively, a falsified response value may be intentionally returned to the attacker to confuse the attacker.

[0221] Here, step **S220** may be performed to determine that validation of the user has succeeded only when the order in which the remote command is requested from the bastion nodes according to the request sequence matches the set request sequence.

[0222] Here, step **S220** may be performed to determine that validation of the user has succeeded only when the order in which the remote command is requested from the bastion nodes according to the request sequence matches the set request sequence within the time window.

[0223] Referring to FIG. 7, a request sequence validation scheme according to an embodiment of the present disclosure may be illustrated.

[0224] The request sequence validation scheme may validate whether an authenticated and authorized user has called a remote command request to bastion nodes according to a set order.

[0225] Because the attacker does not know the order in which bastion nodes present in the bastion group are to be called, it is impossible to intrude into the corresponding bastion node even if access information is merely stolen.

[0226] As shown in FIG. 7, at step **S220**, the valid request sequence is 7, 2, 3, 4, 1, 6, and 5, and it can be seen that validation has succeeded only when bastion nodes with seven numbers designated are to be called in the order of 7, 2, 3, 4, 1, 6, and 5.

[0227] At step **S220**, the order of 1, 2, 3, and 4 or the order of 6, 2, 5, 3, 4, 1, and 7 may be determined to be an invalid request sequence, with the result that the corresponding request may be dropped or, alternatively, a falsified response value may be intentionally returned to the attacker so as to confuse the attacker.

[0228] Further, the method for managing cloud computing infrastructure access based on dynamic parallel nodes according to the embodiment of the present disclosure may forward the remote command at step **S230**.

[0229] That is, at step **S230**, the remote command requested by the validated user may be forwarded to the target compute node included in the computing infrastructure.

[0230] Here, step **S230** may be performed to forward the command depending on a designated validation method so as to perform validation in the bastion node instead of the bastion node, or so as to prevent any problems from occurring at the time of performing validation between bastion nodes.

[0231] Here, at step **S230**, when the command is forwarded, an OS agnostic command conversion unit may perform command conversion as needed.

[0232] Here, step **S230** may be performed to convert the remote command from the user in the format of a command corresponding to a target compute node using information stored in a command mapping database (DB).

[0233] Here, at step **S230**, command conversion may be performed such that the same command can be executed even on the compute nodes having different OSs.

[0234] Here, step **S230** may be performed to handle the request from the user to allow each bastion

node to forward the request to the corresponding compute node by forwarding the request to all related bastion nodes (forward remote commands).

[0235] In the case where compute nodes are based on different OSs or environments when the user requests a remote command from compute nodes in the form of a command working on a specific OS, there is a high likelihood that the corresponding command will not work.

[0236] Referring to FIG. 8, an OS agnostic command conversion process according to an embodiment of the present disclosure is illustrated.

[0237] Here, at step S230, key commands for respective OSs may be configured into an OS agnostic command mapping database (DB), and the command of a source OS may be converted into the command of a target OS based on the OS agnostic command mapping DB.

[0238] Here, step S230 may be performed to provide a command simulator for collecting and analyzing information in the OS agnostic command mapping DB, an external DB or the Internet web and converting the information into a suitable command.

[0239] Here, step S230 may be performed to configure the optimal command and then set up temporary simulators for respective OS environments to proactively validate whether a recommended command can be executed.

[0240] Here, step S230 may be performed to allocate the validated command to each compute node, thus allowing all compute nodes to execute the corresponding command to be OS agnostic.

[0241] Here, step S230 may be performed to allow the user to confirm the converted command in advance and forward the command to the corresponding compute node if necessary.

[0242] Referring to FIG. 8, at step S230, the command simulator may convert Ubuntu 18.04 command into Ubuntu 22.04, Ubuntu 18.04, Windows 2000, and Debian 10 commands based on the OS agnostic command mapping DB.

[0243] Here, at step S230, a feedback handler may update the OS agnostic command mapping DB by feeding back the results of command conversion and the results of command output (stdout, stderr, or the like).

[0244] Here, step S230 may be performed to store and manage the request and result of the remote command by the user, the result of validation, etc. through logging.

[0245] Here, step S230 may be performed to analyze and utilize the result of the request and execution, the validation result, etc. to establish various policies.

[0246] FIG. 10 is a diagram illustrating a computer system according to an embodiment of the present disclosure.

[0247] Referring to FIG. 10, an apparatus for managing cloud computing infrastructure access based on dynamic parallel nodes according to an embodiment of the present disclosure may be implemented in a computer system 1100 such as a computer-readable storage medium. As shown in FIG. 10, the computer system 1100 may include one or more processors 1110, memory 1130, a user interface input device 1140, a user interface output device 1150, and storage 1160, which communicate with each other through a bus 1120. The computer system 1100 may further include a network interface 1170 connected to a network 1180. Each processor 1110 may be a Central Processing Unit (CPU) or a semiconductor device for executing programs or processing instructions stored in the memory 1130 or the storage 1160. Each of the memory 1130 and the storage 1160 may be any of various types of volatile or nonvolatile storage media. For example, the memory 1130 may include Read-Only Memory (ROM) 1131 or Random Access Memory (RAM) 1132.

[0248] An apparatus for managing cloud computing infrastructure access based on dynamic parallel nodes according to an embodiment of the present disclosure may include one or more processors 1110, and memory 1130 configured to store at least one program that is executed by the one or more processors, wherein the at least one program is configured to configure at least two of multiple compute nodes included in a computing infrastructure as representative nodes at preset intervals, validate, by the representative nodes, a user with respect to a remote command requested

by the user for the computing infrastructure, and forward the remote command requested by a validated user to a target compute node included in the computing infrastructure.

[0249] Here, the at least one program may be configured to configure the representative nodes in a multi-layer structure.

[0250] Here, the at least one program may be configured to determine that validation of the user has succeeded only when the remote command is forwarded to the target compute node through the representative nodes configured in the multi-layer structure.

[0251] Here, the at least one program may be configured to set a time window for an accepted request time interval for the remote command with respect to the representative nodes.

[0252] Here, the at least one program may be configured to determine that validation of the user has succeeded only when the remote command is requested from representative nodes within the time window.

[0253] Here, the at least one program may be configured to set an accepted number of requests corresponding to the remote command requested from the representative nodes within the time window.

[0254] Here, the at least one program may be configured to determine that validation of the user has succeeded only when a remote command corresponding to the accepted number of requests is requested from the representative nodes within the time window.

[0255] Here, the at least one program may be configured to set a request sequence related to an order in which the remote command is requested from the representative nodes.

[0256] Here, the at least one program may be configured to determine that validation of the user has succeeded only when the order in which the remote command is requested from the representative nodes depending on the request sequence matches the set request sequence.

[0257] Here, the at least one program may be configured to convert the remote command of the user into a command format corresponding to the target compute node using information stored in a command mapping database.

[0258] The present disclosure may provision and manage a cloud computing infrastructure that utilizes various types of clouds, and improve secure access to the provisioned computing infrastructure.

[0259] Further, the present disclosure may dynamically extend nodes depending on a cloud access level and enhance security by mutually validating the nodes.

[0260] Furthermore, the present disclosure may allow an authenticated and authorized user to freely access nodes and request the nodes to process commands at an external place without passing through a Virtual Private Network (VPN) or a dedicated network.

[0261] As described above, in the apparatus and method for managing cloud computing infrastructure access based on dynamic parallel nodes according to the present disclosure, the configurations and schemes in the above-described embodiments are not limitedly applied, and some or all of the above embodiments can be selectively combined and configured so that various modifications are possible.

Claims

1. An apparatus for managing cloud computing infrastructure access based on dynamic parallel nodes, comprising: one or more processors; and a memory configured to store at least one program that is executed by the one or more processors, wherein the at least one program is configured to: configure at least two of multiple compute nodes included in a computing infrastructure as representative nodes at preset intervals, validate, by the representative nodes, a user with respect to a remote command requested by the user for the computing infrastructure, and forward the remote command requested by a validated user to a target compute node included in the computing infrastructure.

2. The apparatus of claim 1, wherein the at least one program is configured to configure the representative nodes in a multi-layer structure.
3. The apparatus of claim 2, wherein the at least one program is configured to determine that validation of the user has succeeded only when the remote command is forwarded to the target compute node through the representative nodes configured in the multi-layer structure.
4. The apparatus of claim 1, wherein the at least one program is configured to set a time window for an accepted request time interval for the remote command with respect to the representative nodes.
5. The apparatus of claim 4, wherein the at least one program is configured to determine that validation of the user has succeeded only when the remote command is requested from representative nodes within the time window.
6. The apparatus of claim 4, wherein the at least one program is configured to set an accepted number of requests corresponding to the remote command requested from the representative nodes within the time window.
7. The apparatus of claim 6, wherein the at least one program is configured to determine that validation of the user has succeeded only when a remote command corresponding to the accepted number of requests is requested from the representative nodes within the time window.
8. The apparatus of claim 1, wherein the at least one program is configured to set a request sequence related to an order in which the remote command is requested from the representative nodes.
9. The apparatus of claim 8, wherein the at least one program is configured to determine that validation of the user has succeeded only when the order in which the remote command is requested from the representative nodes depending on the request sequence matches the set request sequence.
10. The apparatus of claim 1, wherein the at least one program is configured to convert the remote command of the user into a command format corresponding to the target compute node using information stored in a command mapping database.
11. A method for managing cloud computing infrastructure access based on dynamic parallel nodes, comprising: configuring at least two of multiple compute nodes included in a computing infrastructure as representative nodes at preset intervals; validating, by the representative nodes, a user with respect to a remote command requested by the user for the computing infrastructure; and forwarding the remote command requested by a validated user to a target compute node included in the computing infrastructure.
12. The method of claim 11, wherein configuring as the representative nodes comprises: configuring the representative nodes in a multi-layer structure.
13. The method of claim 12, wherein validating the user comprises: determining that validation of the user has succeeded only when the remote command is forwarded to the target compute node through the representative nodes configured in the multi-layer structure.
14. The method of claim 11, wherein configuring as the representative nodes comprises: setting a time window for an accepted request time interval for the remote command with respect to the representative nodes.
15. The method of claim 14, wherein validating the user comprises: determining that validation of the user has succeeded only when the remote command is requested from representative nodes within the time window.
16. The method of claim 14, wherein configuring as the representative nodes further comprises: setting an accepted number of requests corresponding to the remote command requested from the representative nodes within the time window.
17. The method of claim 16, wherein validating the user comprises: determining that validation of the user has succeeded only when a remote command corresponding to the accepted number of requests is requested from the representative nodes within the time window.

- 18.** The method of claim 11, wherein configuring as the representative nodes comprises: setting a request sequence related to an order in which the remote command is requested from the representative nodes.
- 19.** The method of claim 18, wherein validating the user comprises: determining that validation of the user has succeeded only when the order in which the remote command is requested from the representative nodes depending on the request sequence matches the set request sequence.
- 20.** The method of claim 11, wherein forwarding the remote command comprises: converting the remote command of the user into a command format corresponding to the target compute node using information stored in a command mapping database.
-